WebAssembly vs Docker Gateway Performance Benchmark Study

Supervisors: **N. Modina, S. Secci**

Student: **D. Khvedelidze**

Contents

1 Motivation **3**

2 Goals and Hypotheses **3**
2.1 Goals . . . 3
2.2 Hypotheses . . . 4

3 Experimental Design **4**
3.1 Evaluated Variants . . . 4
3.2 Platform and Environment . . . 5
3.3 Workloads . . . 5
3.4 Benchmark Methodology . . . 6

4 Implementation **6**
4.1 Gateway Architecture . . . 6
4.2 Embedded Wasmtime Caching . . . 7
4.3 Benchmark Scripts . . . 7
4.4 Developments Achieved and Challenges . . . 7

5 Results **8**
5.1 Cold-Start Latency . . . 8
5.2 Warm-Request Latency . . . 9
5.3 Throughput vs Concurrency . . . 11
5.4 Resource Footprint and Efficiency . . . 13

6 Discussion **15**

7 Limitations and Validity Threats **16**

8 Future Work **17**

9 Conclusion **17**

A Additional Throughput Results **18**
A.1 State Workload 18
A.2 Proxy Workload 19

B Latency vs Concurrency **19**

C Resource Profiles by Workload **19**

D Full Throughput Data Tables **19**
D.1 Proxy Workload (RPS) 19
D.2 State Workload (RPS) 19

Abstract

This report presents a reproducible benchmark study comparing the performance overhead of five HTTP gateway deployment variants on a Linux KVM virtual machine. The evaluated execution models are: (1) a native Rust binary executed directly on the host, (2) the same binary inside a Docker container, (3) a gateway that delegates per-request logic to a WebAssembly module via an embedded Wasmtime runtime (in-process), (4) a gateway that spawns a `wasmtime run` CLI subprocess per request, and (5) a gateway that spawns a `wasmedge` CLI subprocess per request. The benchmarks measure cold-start latency, warm-request latency, sustained throughput under varying concurrency, and resource consumption (RSS, CPU). Four workloads are tested: a no-op "hello" endpoint, a CPU-bound SHA-256 chain, an atomic-counter stateful endpoint, and a reverse-proxy forwarding to an upstream service. All gateway code is single-threaded and blocking to isolate runtime overhead from concurrency-framework effects. All results and conclusions are scoped to this specific testbed and should not be generalised without further validation on other platforms and workloads.

1 Motivation

Edge and serverless systems are sensitive to startup latency and per-instance resource footprint. Traditional Linux containers provide mature isolation and packaging but introduce overhead from network namespaces, overlay filesystems, and (on non-Linux hosts) a virtualisation layer. WebAssembly (Wasm) modules offer an alternative execution model that promises sub-millisecond instantiation and smaller memory use through a sandboxed, capability-based runtime.

This project evaluates those claims empirically by constructing a testbed that compares native, containerised, and Wasm-based HTTP gateway middleware under controlled conditions. The contribution is a reproducible pipeline and a dataset from which mechanistic explanations of observed overhead can be drawn.

2 Goals and Hypotheses

2.1 Goals

- Measure cold-start latency (process launch to first HTTP 200) for five deployment variants.
- Measure warm-request latency at the p50, p90, and p99 percentiles.
- Compare sustained throughput (requests/second) across four workloads at seven concurrency levels.
- Quantify resource consumption (RSS, CPU, CPU per 1 000 RPS) per variant.
- Produce a fully automated, reproducible benchmark pipeline with timestamped outputs and machine-environment snapshots.

2.2 Hypotheses

1. Native execution should exhibit the lowest cold-start and highest throughput because it incurs no runtime indirection.
2. Docker cold start should be substantially slower than native due to container-image initialisation and network-namespace setup.
3. The embedded Wasmtime variant should approach native warm-latency because it avoids subprocess creation.
4. CLI-based Wasm variants (wasmtime, wasmedge) should exhibit throughput ceilings proportional to their per-request process-spawn cost.
5. For CPU-bound workloads, the invocation overhead of all variants should become negligible relative to the SHA-256 computation cost.

3 Experimental Design

3.1 Evaluated Variants

Five deployment variants are evaluated. Table 1 describes the execution model and what each variant primarily measures.

Variant	Execution model	What it measures
`native_local`	Single Rust binary, direct process execution on the host.	Zero-overhead baseline.
`native_docker`	Same binary inside a Docker container (Linux, no Desktop VM).	Container isolation overhead: network namespace, overlay FS, cgroup scheduling.
`wasm_host_wasmtime_embedded`	Gateway links the Wasmtime crate (v41) in-process. The Wasm module is compiled once; each request instantiates a new `Store` + WASI context.	In-process Wasm instantiation overhead per request.
`wasm_host_wasmtime`	Gateway spawns `wasmtime run` as a child process per request, piping request/response via stdin/stdout.	Process-spawn + runtime-initialisation overhead (not intrinsic Wasm execution performance).
`wasm_host_cli`	Same subprocess model, but using the `wasmedge` CLI.	Process-spawn + WasmEdge runtime-initialisation overhead.

Table 1: Evaluated deployment variants and what each primarily measures.

Critical distinction. The two CLI-based Wasm variants fork a new operating-system process for every HTTP request. Their throughput ceiling is therefore determined by process-spawn overhead, not by the steady-state execution speed of WebAssembly. These variants represent a worst-case isolation model and should not be conflated with in-process Wasm execution.

3.2 Platform and Environment

Parameter	Value
Host OS	Linux 5.4.0-1032-kvm (Ubuntu, x86_64)
CPU	4 cores (KVM virtual machine)
RAM	15 GiB total, ≈13 GiB available
Rust	1.93.1 (2026-02-11)
Wasmtime	v41 (embedded crate) + CLI
WasmEdge	latest CLI
Docker	Linux native (no Desktop VM)
`wrk`	4.2.0
`hyperfine`	1.20.0

Table 2: Testbed environment. Source: `results/meta/env_snapshot_20260222_195454.txt`.

All benchmarks run on loopback (`127.0.0.1`). The Docker daemon runs natively on Linux; there is no Docker Desktop virtualisation layer, unlike the mid-term setup on macOS.

3.3 Workloads

Four workloads exercise different cost profiles:

Workload	Endpoint	Description
hello	`GET /`	Returns `"hello"` (or `"wasm:hello"` for Wasm variants). No-op baseline.
compute	`GET /compute?iters=20000`	Chains 20 000 SHA-256 iterations. CPU-bound.
state	`GET /state`	Atomic counter (`AtomicU64::fetch_add`). Stateful request handling.
proxy	`GET /<any>`	Forwards to `http-echo` upstream on port 18080. Network-I/O-bound.

Table 3: Benchmark workloads.

In Wasm variants the response body passes through a trivial Wasm transform (prepend `"wasm:"`) to isolate invocation mechanism overhead from application logic cost. The Wasm module (`gateway_wasm`) has zero dependencies and targets `wasm32-wasip1`.

3.4 Benchmark Methodology

Cold start. Measured with `hyperfine --warmup 0 --runs 20`. Each iteration:

1. Kill any stale listener on port 18081.
2. Spawn the gateway process in the background.
3. Poll `/health` (up to 600 attempts at 10 ms intervals).
4. Fire one `curl` request to `/`.
5. `kill -9` the process.

Three benchmark repeats yield $N = 60$ samples per variant.

Warm latency. Measured with `hyperfine --warmup 20 --runs 300`. The gateway is started once and kept alive. Each iteration runs `curl -fsS http://127.0.0.1:18081/`. The measurement includes `curl` process startup overhead (≈8 ms), so absolute values are inflated; *relative* differences between variants are the meaningful signal.

Throughput. Measured with `wrk` (4 threads) at seven concurrency levels: 10, 50, 100, 200, 400, 800, 1200 connections. Each run lasts 30 s. A Python `psutil`-based sampler records RSS and CPU at 200 ms intervals throughout.

Data pipeline. `scripts/bench_all.sh` orchestrates all three benchmark types with configurable repeats. Raw data are written to timestamped files in `results/`. Summary statistics are computed into `results/summary/` (percentile CSVs) and `results/aggregated/throughput.csv`. Plots are generated in `results/plots/`. A machine-environment snapshot is captured in `results/meta/`.

4 Implementation

4.1 Gateway Architecture

The project is a Rust workspace with three crates:

- `gateway_native` — single-threaded, blocking TCP gateway. Handles all four workloads directly using `std::net::TcpStream`. No external dependencies at runtime.
- `gateway_host` — same TCP gateway, but delegates a response-body transform to a Wasm module. The actual compute (SHA-256) and network I/O (proxy) occur in the host; only the response body passes through the Wasm boundary. Supports three runtime modes selected via `WASM_RUNTIME`: `wasmedge` (CLI), `wasmtime` (CLI), `wasmtime_embedded` (in-process).

- `gateway_wasm` — minimal WASI module: reads stdin, prepends `"wasm:"`, writes to stdout.

No async. All gateway code is deliberately single-threaded with blocking I/O and explicit read/write timeouts. This eliminates confounding factors from async runtime scheduling (e.g., Tokio) and keeps the comparison fair across all variants.

4.2 Embedded Wasmtime Caching

The `wasmtime_embedded` variant compiles the Wasm module once at first use and caches the `Engine` + `Module` pair in a process-global `Lazy<RwLock<HashMap>>`. Each subsequent request allocates a fresh `Store` and WASI context (`MemoryInputPipe` / `MemoryOutputPipe`), instantiates the module, calls `_start`, and reads the output pipe. This amortises compilation cost across all requests while maintaining per-request isolation.

4.3 Benchmark Scripts

- `scripts/bench_cold_start.sh` — cold-start measurement.
- `scripts/bench_warm_latency.sh` — warm single-request latency.
- `scripts/bench_throughput.sh` — throughput with resource sampling.
- `scripts/bench_all.sh` — orchestrator; captures environment snapshot, runs all benchmarks with configurable repeats.
- `scripts/run_native_local.sh`, `scripts/run_docker.sh`, `scripts/run_wasm_host_local.sh` — per-variant launchers.

4.4 Developments Achieved and Challenges

1. **Fully automated pipeline**: a single `scripts/bench_all.sh` invocation builds all binaries, runs all three benchmark suites across all five variants, computes summary statistics, and generates plots.
2. **Cross-platform port management**: scripts implement six detection methods (`lsof`, `fuser`, `ss`, `nc`, `/dev/tcp`, Python `/proc/net/tcp`) for portable process cleanup.
3. **Resource sampler**: a continuous `psutil`-based sampler at 200 ms intervals is sliced by timestamp window per `wrk` run.
4. **Docker network auto-detection**: `scripts/run_docker.sh` detects the Docker Compose network so the containerised gateway resolves `upstream` by hostname.

5. **Challenge — accept-queue saturation**: single-threaded `accept()` causes throughput instability at >200 connections for `native_local`. This is a testbed artefact, not a runtime limitation.

6. **Challenge — sampler resolution**: the 200 ms sampling interval is too coarse to capture the RSS and CPU of CLI subprocess Wasm runtimes that live for only a few milliseconds per request.

5 Results

All numeric values below are taken directly from the CSV files listed in the source column. No values are rounded beyond the precision present in those files.

5.1 Cold-Start Latency

Cold start measures the wall-clock time from `exec` of the gateway binary to the first successful HTTP 200 response. Each variant was measured 20 times per repeat, with 3 repeats, yielding $N = 60$ samples per variant.

Variant	**p50 (ms)**	**p90 (ms)**	**p99 (ms)**	N
`native_local`	176.36	192.18	202.64	60
`wasm_host_wasmtime`	190.29	207.09	214.74	60
`wasm_host_cli`	203.49	215.52	231.36	60
`wasm_host_wasmtime_embedded`	232.58	250.30	258.22	60
`native_docker`	3143.39	4426.92	5499.42	60

Table 4: Cold-start latency percentiles. Source: `results/summary/cold_start_percentiles.csv`.

Interpretation. In this testbed, all non-Docker variants start in under 260 ms at the p99 level. The native baseline is 176.36 ms (p50). The two CLI Wasm variants add 13.93 ms (wasmtime) and 27.13 ms (wasmedge) respectively. The embedded Wasmtime variant is the slowest non-Docker starter at 232.58 ms (p50) because its first invocation includes one-time Wasm module compilation.

Docker cold start is an order of magnitude higher at 3143.39 ms (p50), with high variance (p99 at 5499.42 ms). This includes container-image layer extraction, network-namespace creation, cgroup setup, and overlay-filesystem mounting—costs that do not apply to bare-process or in-process Wasm execution.

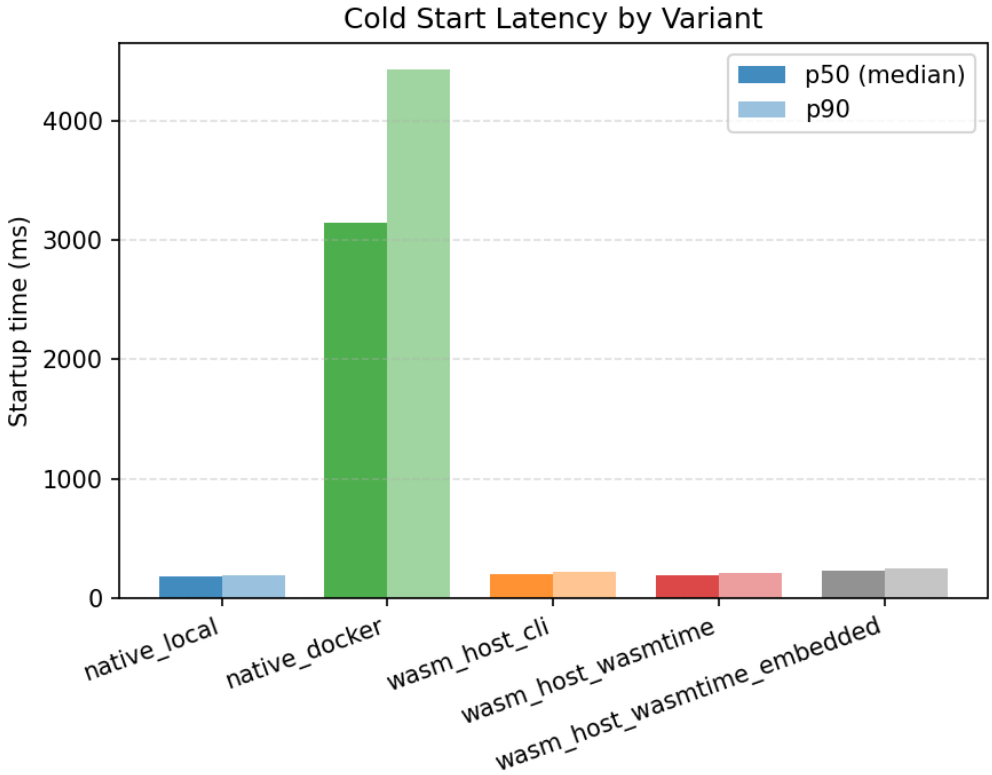

Figure 1: Cold-start latency (p50 and p90) by variant. Source: `results/plots/cold_start_median_p90.png`.

5.2 Warm-Request Latency

Warm latency measures the `curl` round-trip time to a running gateway for the `hello` workload. The gateway is started once and kept alive across 300 measured iterations (after 20 warmup iterations).

Interpretation. The absolute values include approximately 8 ms of `curl` process-startup overhead; therefore, only *relative* differences are meaningful.

Native local, Docker, and embedded Wasm cluster within 0.59 ms of each other at p50 (8.79 vs 9.17 vs 9.38 ms). This indicates that, in steady state on this testbed, container networking overhead and in-process Wasm instantiation cost are both negligible for a no-op handler.

The CLI Wasm variants show 11.61 ms (wasmtime) and 17.99 ms (wasmedge) additional latency at p50 relative to native. This overhead is entirely attributable to forking a child process and initialising the Wasm runtime per request.

Tail latency (p99) follows the same ranking. Native local shows 13.88 ms; the embedded variant tracks closely at 14.71 ms; CLI variants reach 26.54 ms and 33.97 ms.

Variant	**p50 (ms)**	**p90 (ms)**	**p99 (ms)**	*N*
`native_local`	8.79	10.58	13.88	600
`native_docker`	9.17	11.08	14.66	300
`wasm_host_wasmtime_embedded`	9.38	11.07	14.71	300
`wasm_host_wasmtime`	20.40	22.61	26.54	300
`wasm_host_cli`	26.78	30.11	33.97	300

Table 5: Warm-request latency percentiles (`hello` workload, steady state). Source: `results/summary/warm_latency_percentiles.csv`.

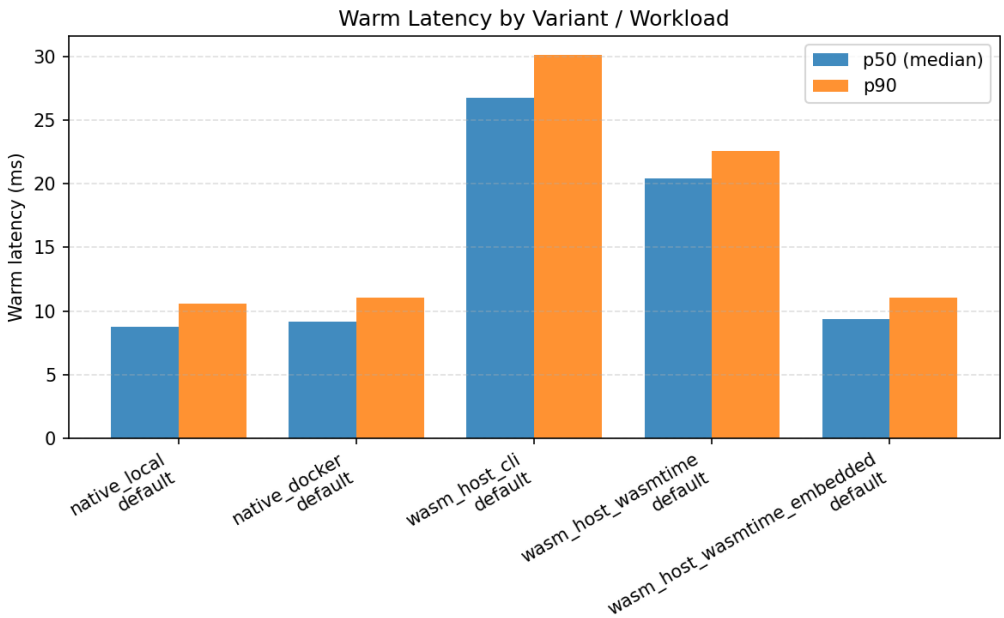

Figure 2: Warm-request latency (p50 and p90) by variant. Source: `results/plots/warm_latency_median_p90.png`.

5.3 Throughput vs Concurrency

Throughput is measured with `wrk` (4 threads, 30 s per run) at seven concurrency levels: 10, 50, 100, 200, 400, 800, and 1200 connections. Each workload × variant × concurrency combination is a single 30 s run.

5.3.1 Hello Workload (No-Op)

Conns	native_local	native_docker	wasm_embedded	wasm_wasmtime	wasm_cli
10	19 868	6 453	3 476	101	58
50	21 988	8 044	3 488	101	57
100	22 745	8 696	3 569	100	57
200	22 442	8 784	1 406	98	55
400	4 566	9 033	1 380	96	15
800	6 647	9 114	1 323	95	54
1200	5 618	8 898	2 230	96	12

Table 6: Hello workload throughput (RPS). Source: `results/summary/throughput_summary.csv`, rows where `workload=hello`.

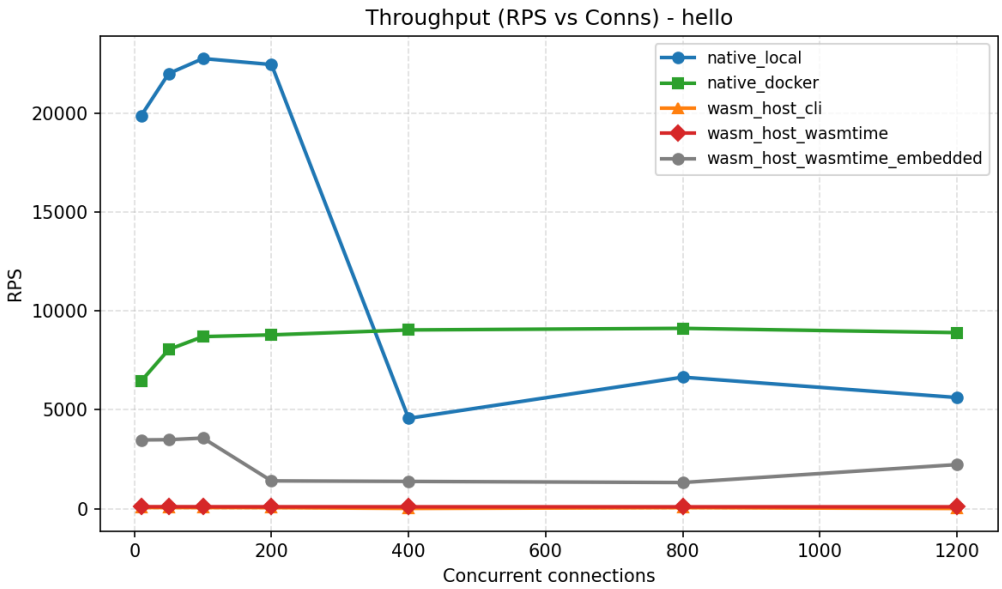

Figure 3: Hello workload: throughput vs concurrency. Source: `results/plots/throughput_hello.png`.

Interpretation. Native local peaks at 22 745 RPS at 100 connections, then drops sharply at 400+ connections. This degradation is an artefact of the single-threaded `accept()` queue saturating on this 4-core machine: when more connections arrive than the gateway can drain in one thread, queueing delays dominate. Docker stabilises around 8 700–9 100 RPS and does *not* exhibit the same drop, likely because the container's network namespace introduces a pacing effect that prevents accept-queue saturation.

Embedded Wasm achieves 3 569 RPS at 100 connections (15.7% of native peak), with per-request `Store` instantiation as the bottleneck. The CLI variants are capped at ≈100 RPS (wasmtime) and ≈57 RPS (wasmedge), entirely limited by the time to fork and initialise a child process per request.

5.3.2 Compute Workload (CPU-Bound)

Conns	native_local	native_docker	wasm_embedded	wasm_wasmtime	wasm_cli
10	23	135	131	13	11
50	136	134	132	56	40
100	135	133	130	55	39
200	133	132	87	53	37
400	132	107	125	52	36
800	132	130	129	14	24
1200	132	57	95	53	23

Table 7: Compute workload throughput (RPS). Source: `results/summary/throughput_summary.csv`, rows where `workload=compute`.

Interpretation. At 100 connections, native (135 RPS), Docker (133 RPS), and embedded Wasm (130 RPS) converge within 3.7% of each other. The 20 000-iteration SHA-256 chain (≈7.3 ms per request) dominates execution time, making the Wasm instantiation overhead negligible. This confirms Hypothesis 5: for CPU-bound workloads on this testbed, the execution model overhead amortises.

CLI variants remain lower (55 and 39 RPS at 100 connections) because process-spawn cost adds to each request's computation time.

The anomalous result at {native_local, compute, 10 connections} (23 RPS vs 135 RPS at 50) is likely a warm-up artefact: with only 10 connections the single-threaded gateway may not reach full CPU utilisation within the measurement window.

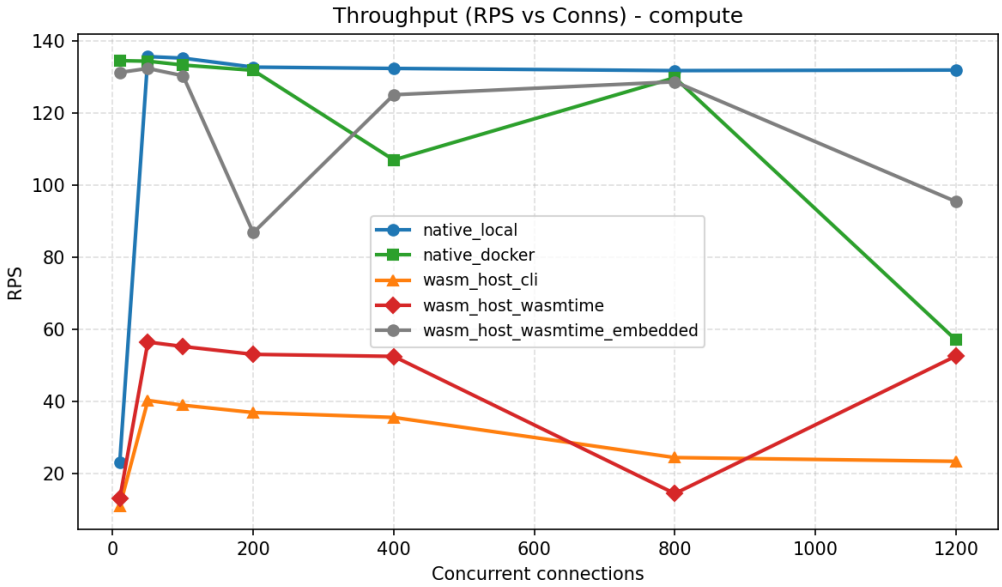

Figure 4: Compute workload: throughput vs concurrency. Source: `results/plots/throughput_compute.png`.

5.4 Resource Footprint and Efficiency

Resource consumption is sampled continuously during throughput runs at 200 ms intervals.

Variant	RSS (KB)	CPU avg (%)	CPU/1k RPS
`native_local`	788	90.31	3.97
`native_docker`	1 180	41.67	4.79
`wasm_embedded`	19 760	99.58	27.90
`wasm_wasmtime`	4 836	2.83	28.39
`wasm_cli`	4 692	2.03	35.88

Table 8: Resource footprint during `hello` workload at 100 connections. Source: `results/summary/throughput_summary.csv`, rows where `workload=hello`, `conns=100`. CPU/1k RPS = `cpu_per_1k_rps` column.

Memory. Native local uses only 788 kB. Docker adds ≈392 kB for container metadata (1 180 KB total). The CLI Wasm variants use ≈4.7 MB (host process only; the short-lived child processes are not captured by the 200 ms sampler). Embedded Wasm uses 19.3 MB, reflecting the compiled Wasm module, Wasmtime engine, and per-request `Store` allocations kept in process memory.

CPU efficiency. CPU per 1 000 RPS normalises CPU usage by throughput, allowing cross-variant comparison of how much CPU is "spent" per unit of useful work. Native local and Docker are the most CPU-efficient at 3.97 and 4.79 respectively. Embedded Wasm is 7× less efficient at 27.90 because each request incurs Wasm module instantiation and WASI pipe I/O. CLI variants show low *host-process* CPU (2–3%) because the actual work occurs in forked child processes not tracked by the host-process sampler.

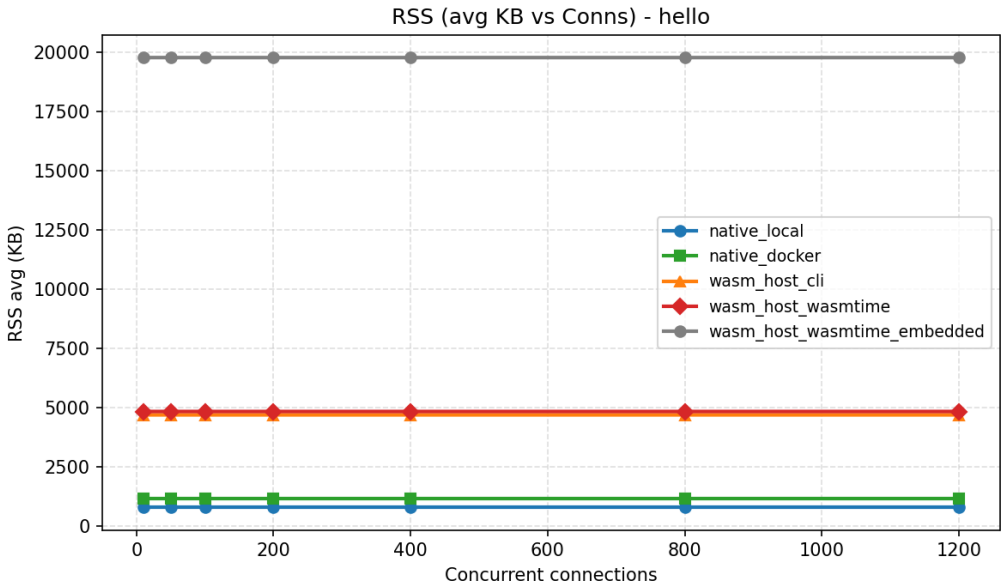

Figure 5: Gateway RSS during hello workload by variant and concurrency. Source: `results/plots/rss_hello.png`.

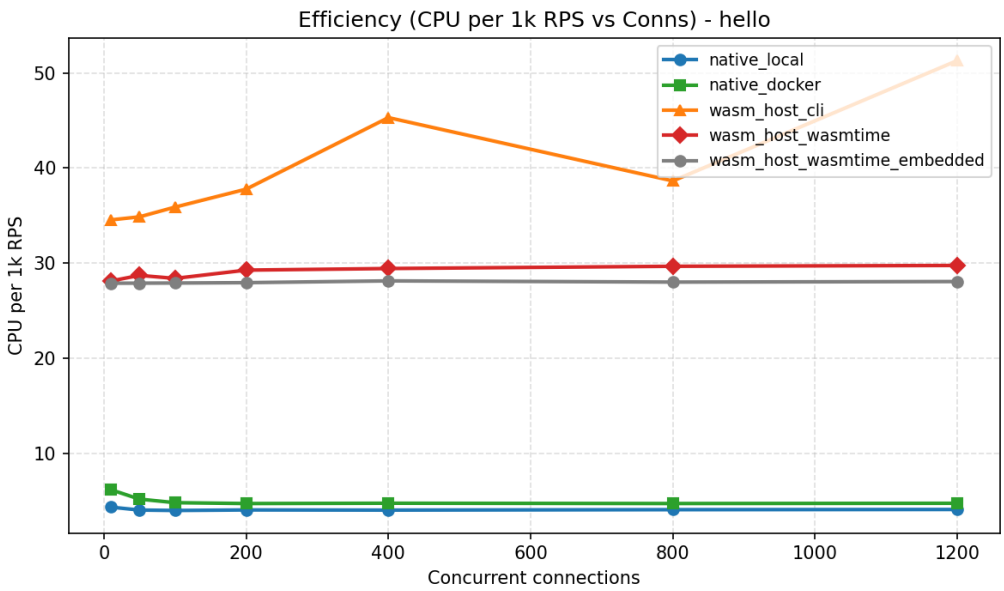

Figure 6: CPU efficiency (CPU% per 1k RPS) for hello workload. Source: `results/plots/efficiency_hello.png`.

6 Discussion

Cold start. All Wasm variants start within 56 ms of the native baseline at the p50 level. Docker cold start is 17.8× slower than native, dominated by container-image and network-namespace initialisation. For latency-sensitive cold-start scenarios on this testbed, any of the Wasm variants outperforms Docker by an order of magnitude.

Warm latency. In steady state, Docker and embedded Wasm add less than 1 ms over native at p50. The differences are within measurement noise given the ≈8 ms curl overhead floor. CLI Wasm adds 12–18 ms per request due to process-creation cost. This confirms that in-process Wasm instantiation is a viable low-overhead mechanism for per-request isolation on this testbed.

Throughput. The hello workload reveals a 6.4× gap between native and embedded Wasm at 100 connections (22 745 vs 3 569 RPS). For CPU-bound work (compute at 100 connections), the gap shrinks to 3.7% (135 vs 130 RPS) because the computation dominates. For I/O-bound work (proxy at 100 connections), embedded Wasm reaches 839 RPS vs 1 138 RPS native (73.7%). The ratio of gateway overhead to workload cost determines whether Wasm is a practical choice for a given scenario.

Resource trade-offs. Embedded Wasm uses 25× more memory than native (19 760 vs 788 KB) but offers per-request sandboxing. The CPU efficiency gap (27.9 vs 4.0 CPU%/1k RPS for hello) reflects the per-request instantiation cost. For compute workloads this gap narrows (767.0

vs 738.7), again confirming that workload-dominated scenarios amortise the overhead.

No universal claim. These results do not support a general claim that "Wasm is faster than Docker" or vice versa. The performance ranking depends on the metric (cold start favours Wasm; steady-state throughput favours native), the workload (CPU-bound work equalises), and the platform (Linux native Docker vs macOS Docker Desktop would yield different results).

7 Limitations and Validity Threats

1. **Loopback networking.** All communication uses `127.0.0.1`, eliminating real network latency and jitter. Results may differ on a multi-host deployment with physical or virtual NICs.
2. **Single-host load generation.** `wrk` runs on the same machine as the gateway under test, competing for the same 4 CPU cores. At high concurrency, load-generator scheduling may interfere with gateway throughput.
3. **KVM virtualisation.** The testbed is a KVM virtual machine. CPU scheduling and memory allocation pass through the hypervisor, which may add latency variance not present on bare metal.
4. **Single-threaded gateway.** The blocking, single-threaded design limits maximum throughput and causes accept-queue saturation at high concurrency. This is a deliberate design choice for fairness, but it means results are not directly transferable to production async gateways.
5. **Curl overhead in warm latency.** Each warm-latency iteration spawns a `curl` process ($\approx$8 ms overhead), inflating absolute values. Only relative comparisons are meaningful.
6. **Resource sampler resolution.** The 200 ms sampling interval cannot capture the RSS or CPU of CLI subprocess Wasm runtimes, which live for only a few milliseconds per request. The reported host-process CPU (2–3%) underestimates total system cost for CLI variants.
7. **Sample counts.** Cold start: $N = 60$; warm latency: $N = 300$–600; throughput: single 30 s run per configuration. These counts may not capture rare tail-latency events or account for long-term drift.
8. **Trivial Wasm module.** The transform module prepends a string. A more complex Wasm plugin with memory allocation, I/O, or multi-function calls would increase per-request overhead.
9. **Wasmtime JIT caching.** The embedded variant compiles the module once and reuses compiled code. Cold start for the embedded variant therefore includes this compilation, but subsequent requests benefit from caching. A scenario requiring frequent module reloads would show different characteristics.

10. **Docker on Linux vs macOS.** The mid-term report used Docker Desktop on macOS (which adds a Linux VM layer). This final testbed uses native Linux Docker, so Docker overhead here is lower than on macOS. Results should not be compared directly across platforms.

8 Future Work

- Run benchmarks on bare-metal hardware to eliminate hypervisor variability.
- Replace `curl`-based warm-latency measurement with a persistent-connection client to remove process-spawn overhead from the measurement.
- Add a multi-threaded gateway variant to test scalability beyond single-thread bottlenecks.
- Benchmark a non-trivial Wasm plugin (e.g., JSON transformation, request routing with regex) to evaluate overhead with realistic application logic.
- Compare Wasmtime's ahead-of-time (AOT) compilation mode against the current JIT-on-first-use approach.
- Introduce a Firecracker or gVisor baseline for lightweight VM comparison.
- Increase sample counts and add multiple-run throughput measurements to improve statistical confidence.

9 Conclusion

This project delivers a reproducible benchmark suite for comparing native, containerised, and WebAssembly-based HTTP gateway middleware on a Linux KVM testbed with 4 CPU cores and 15 GiB RAM.

The principal findings, *scoped to this testbed*, are:

1. **Cold start:** All Wasm variants add 14–56 ms over the native baseline (176 ms p50). Docker is 17.8× slower at 3 143 ms p50.
2. **Warm latency:** Docker and embedded Wasm are within 1 ms of native in steady state. CLI Wasm adds 12–18 ms per request due to process spawning.
3. **Throughput (no-op):** Native peaks at 22 745 RPS; Docker at 9 114; embedded Wasm at 3 569; CLI variants below 101 RPS.
4. **Throughput (CPU-bound):** Native, Docker, and embedded Wasm converge at ≈130 RPS—the SHA-256 computation dominates, making runtime overhead negligible.
5. **Resources:** Embedded Wasm uses 25× more memory than native (19.3 MB vs 0.8 MB) and 7× more CPU per unit of throughput for no-op work.

No universal "Wasm is faster than Docker" claim is supported. The performance trade-off depends on the workload profile: for CPU- or I/O-bound tasks where computation dominates gateway overhead, embedded Wasm provides lightweight per-request isolation at near-native speed. For routing-only workloads, native execution remains an order of magnitude faster. CLI-based Wasm invocation measures process-spawn overhead and is not representative of production Wasm deployment performance.

References

- Wasmtime: `https://github.com/bytecodealliance/wasmtime`
- WasmEdge: `https://github.com/WasmEdge/WasmEdge`
- Docker: `https://www.docker.com`
- wrk: `https://github.com/wg/wrk`
- hyperfine: `https://github.com/sharkdp/hyperfine`

A Additional Throughput Results

A.1 State Workload

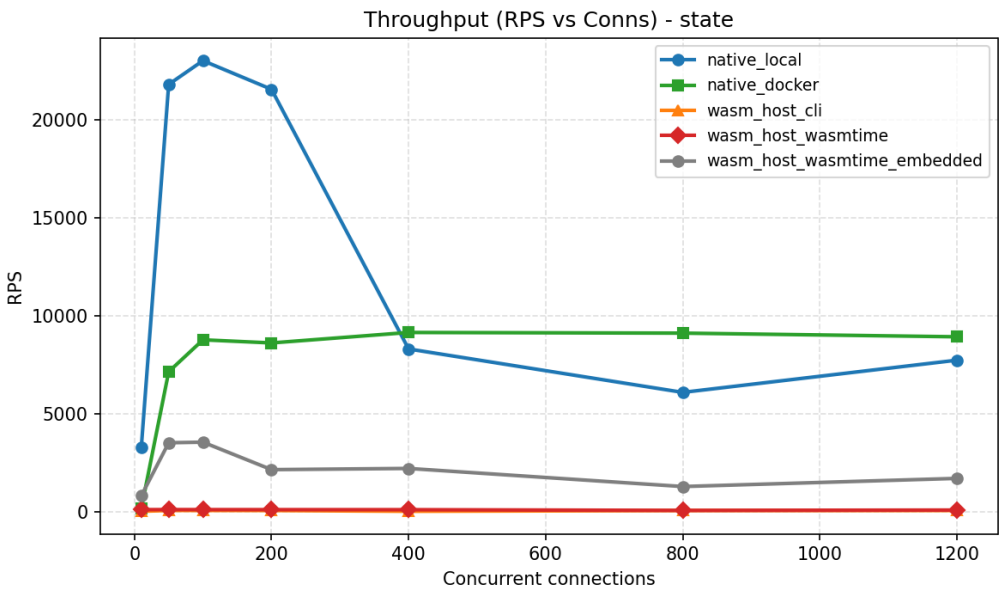

Figure 7: State workload: throughput vs concurrency. Source: `results/plots/throughput_state.png`.

A.2 Proxy Workload

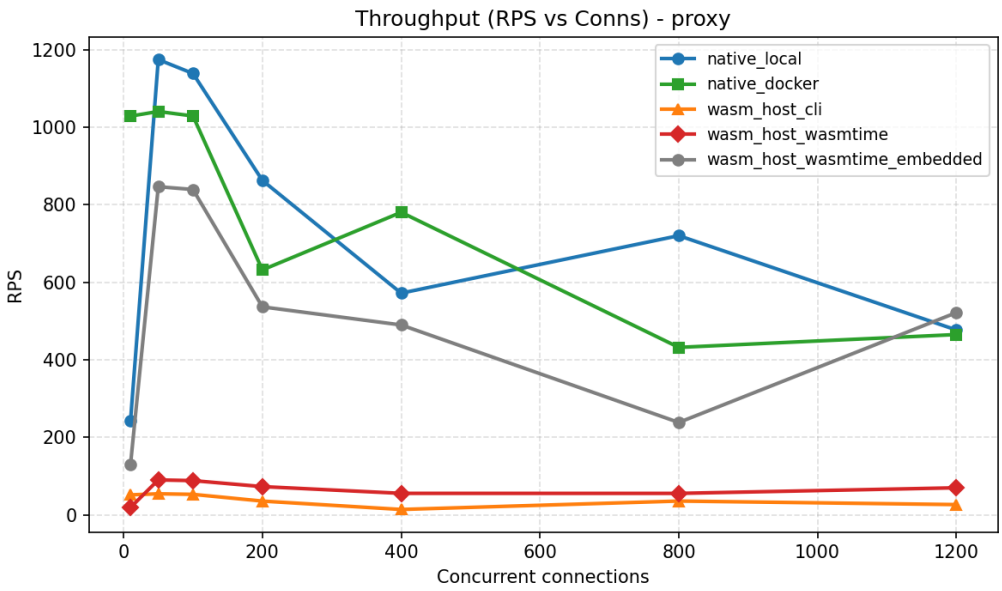

Figure 8: Proxy workload: throughput vs concurrency. Source: `results/plots/throughput_proxy.png`.

B Latency vs Concurrency

C Resource Profiles by Workload

D Full Throughput Data Tables

D.1 Proxy Workload (RPS)

D.2 State Workload (RPS)

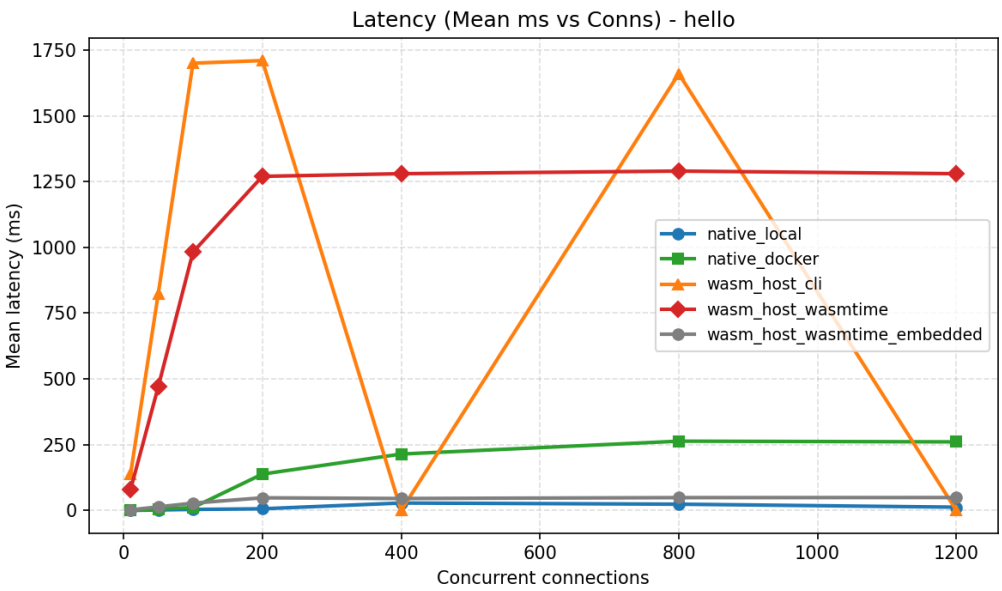

Figure 9: Hello workload: mean latency vs concurrency. Source: `results/plots/latency_hello.png`.

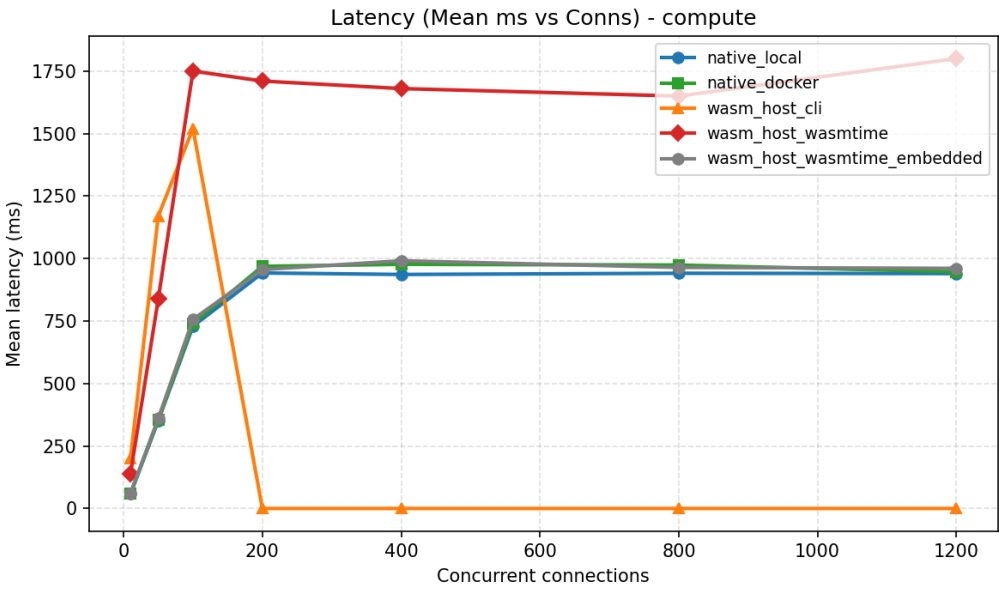

Figure 10: Compute workload: mean latency vs concurrency. Source: `results/plots/latency_compute.png`.

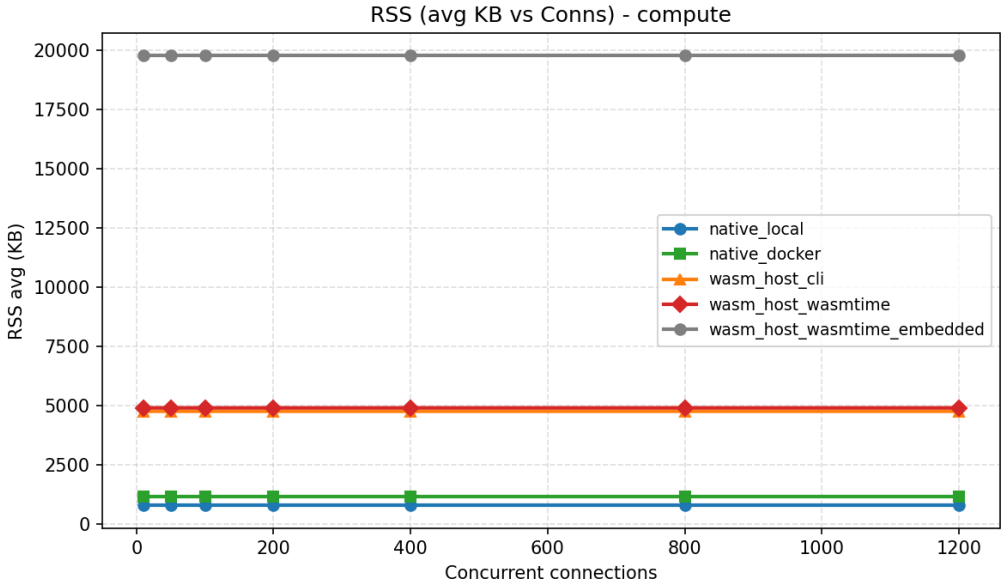

Figure 11: RSS during compute workload. Source: `results/plots/rss_compute.png`.

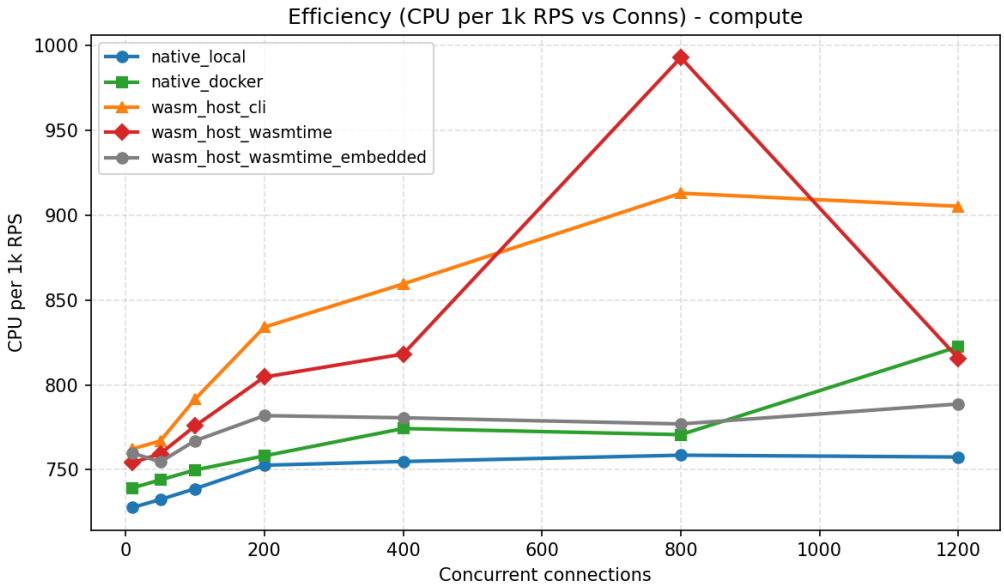

Figure 12: CPU efficiency during compute workload. Source: `results/plots/efficiency_compute.png`.

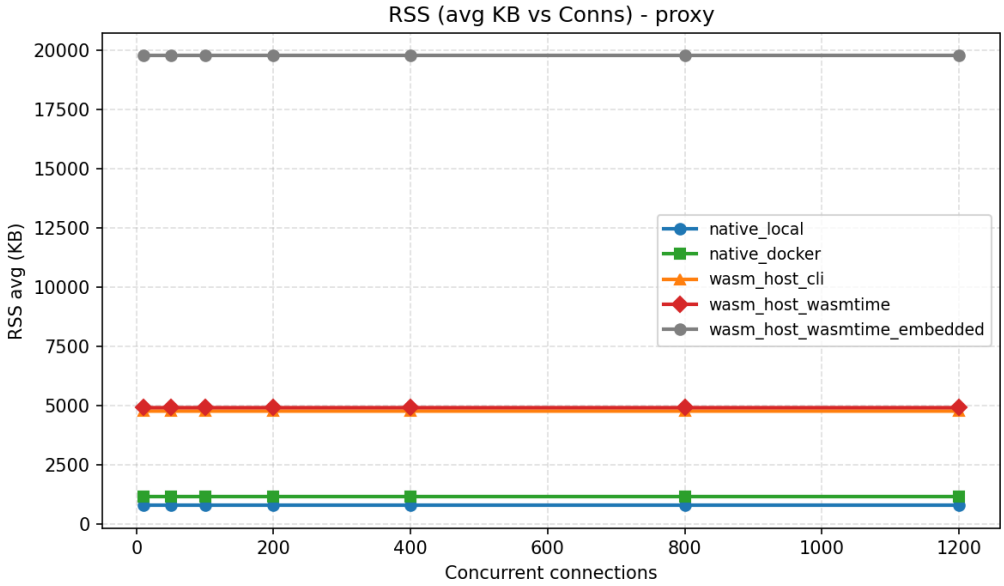

Figure 13: RSS during proxy workload. Source: `results/plots/rss_proxy.png`.

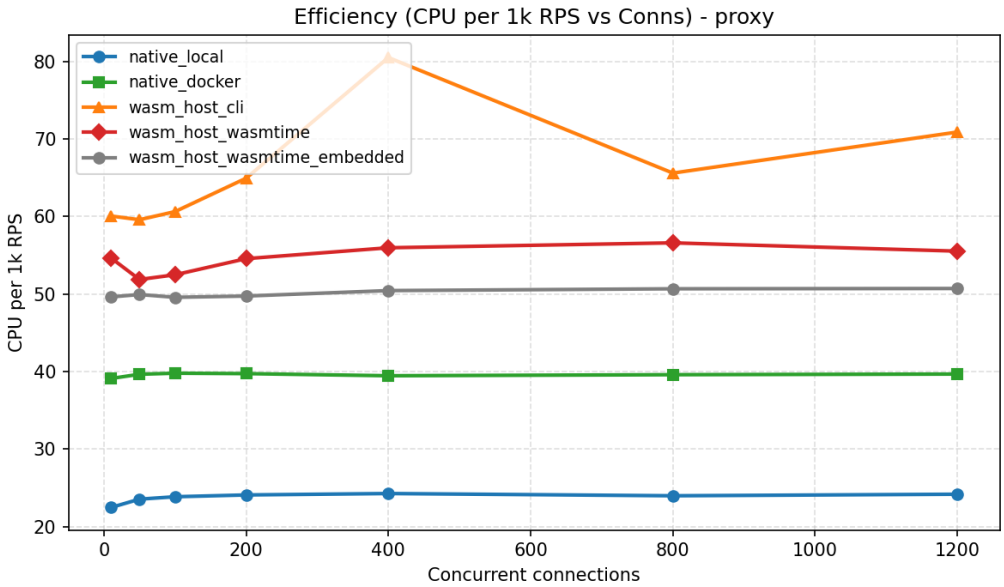

Figure 14: CPU efficiency during proxy workload. Source: `results/plots/efficiency_proxy.png`.

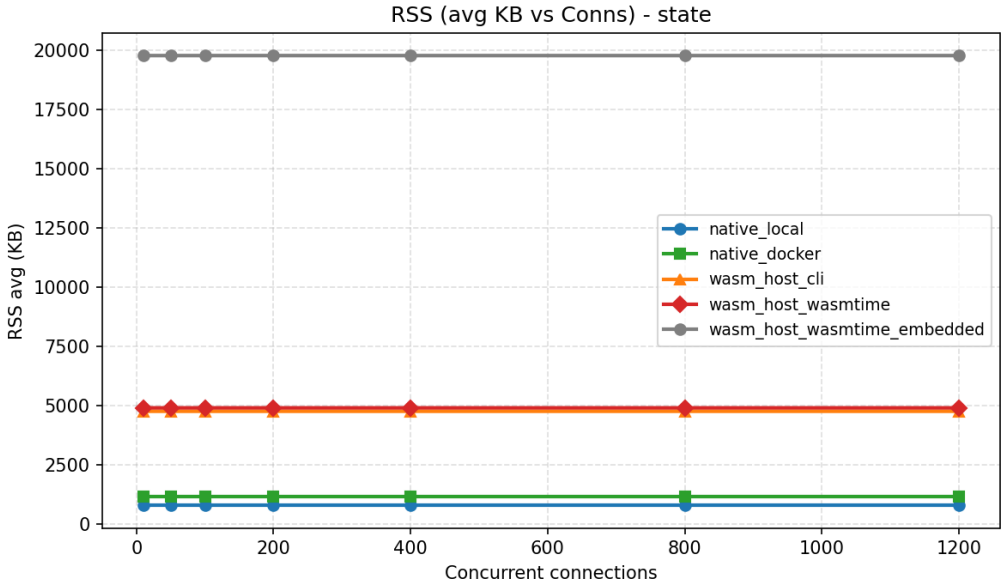

Figure 15: RSS during state workload. Source: `results/plots/rss_state.png`.

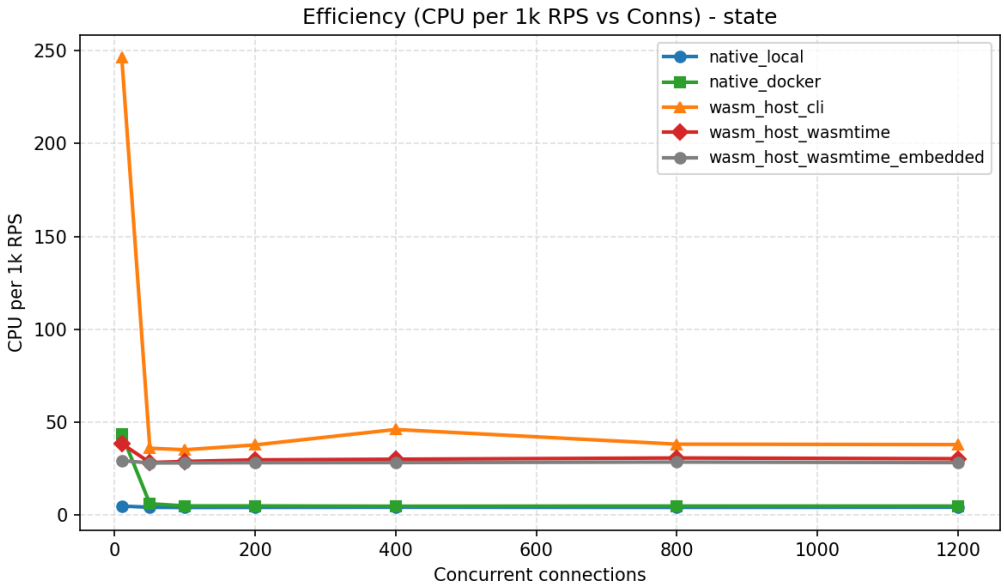

Figure 16: CPU efficiency during state workload. Source: `results/plots/efficiency_state.png`.

Conns	native_local	native_docker	wasm_embedded	wasm_wasmtime	wasm_cli
10	242	1 028	130	19	51
50	1 174	1 040	846	90	54
100	1 138	1 029	839	88	53
200	863	632	536	73	35
400	572	780	490	55	14
800	720	432	238	55	36
1200	477	465	522	70	26

Table 9: Proxy workload throughput (RPS). Source: `results/summary/throughput_summary.csv`, rows where `workload=proxy`.

Conns	native_local	native_docker	wasm_embedded	wasm_wasmtime	wasm_cli
10	3 263	146	801	99	14
50	21 789	7 133	3 511	101	58
100	23 000	8 761	3 542	100	56
200	21 538	8 602	2 139	98	54
400	8 294	9 136	2 200	97	15
800	6 080	9 103	1 278	60	53
1200	7 723	8 916	1 693	78	54

Table 10: State workload throughput (RPS). Source: `results/summary/throughput_summary.csv`, rows where `workload=state`.